

MEDSPHERE

AD23731 Foundations of Agentic AI - Mini-Project

Submitted by

Surweesh SP 231501165

Vasanth Kumar P 231501179

Santhosh Kumar S 231501147

Department of Artificial Intelligence and Machine Learning



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

SEPTEMBER 2026



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this mini-project reportMEDSPHERE... is the bonafide work of ...SURWEESH SP 231501165; VASANTHA KUMAR 231501179; SANTHOSH KUMAR S 231501147..... VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in-charge

Submitted to mini-project viva voce examination held on:

INTERNAL EXAMINER

EXTERNAL EXAMINE

Table of Contents

1.	Introduction	4
2.	Problem Statement and Business Case	5
3.	User and Stakeholder Analysis	6
4.	Proposed Agentic AI Solution	7
5.	Dataset / Knowledge Sources	8
6.	Tools and Technologies Used	10
7.	Agent Architecture	11
8.	Agent Workflow and Orchestration	13
9.	Agentic RAG Implementation	15
10.	System Implementation	17
11.	Experimental Evaluation and Results	18
12.	Comparison, Innovation and Scalability	21
13.	Conclusion and Future Work	23

1. Introduction

1.1 Background and Motivation

Pharmaceutical and life-science organisations rely on **Key Opinion Leaders (KOLs)** recognised healthcare experts whose research, clinical experience and professional standing shape how therapies are understood and adopted within a specialty. Medical Affairs teams and Medical Science Liaisons (MSLs) must decide which experts to engage for advisory boards, clinical-trial collaboration, scientific exchange and evidence generation. Getting this choice wrong wastes scarce engagement resources and can leave the most relevant experts untouched.

The evidence needed for this decision is scattered across many independent sources: publication databases, citation indices, clinical-trial registries, researcher-identity services and provider registries. Each capture one facet of a healthcare professional (HCP), yet an expert's true standing depends on the combination of research output, disease-specific expertise, institutional position and – critically – **the network of collaborators and referral relationships** around them. Viewed one indicator at a time, a KOL's influence is easy to mis-measure.

Graph technology is a natural fit. HCPs, institutions, publications, trials and diseases are entities; co-authorship, affiliation, trial participation and expertise are relationships. A graph captures this structure directly, graph neural networks (GNNs) can learn from it, and large language models (LLMs) can make it accessible through plain-language questions and explanations.

1.2 Need for Agentic AI

A single-shot chatbot or a static dashboard is not sufficient for KOL discovery. A query such as “Find the top influential oncology HCPs in Chennai” or “Why is Dr. T. Smith ranked highly?” cannot be answered by one model call, because answering it requires a **sequence of decisions and actions** over structured data:

- **Interpreting intent** – is the user searching, comparing, profiling or asking for an explanation? Which entities, filters and ranking criteria are involved?
- **Resolving ambiguity** – several HCPs may share a name; the system must ask the human which one is meant rather than guess.
- **Choosing and using tools** – translating language into a graph query, executing it against Neo4j, and retrieving the ranked HCPs, relationship paths, trials and expertise evidence.
- **Checking its own work** – validating the generated query and retrying when it is invalid, instead of returning a wrong or hallucinated answer.
- **Grounded synthesis and human oversight** – explaining results using retrieved evidence and letting a person accept, reject or trigger recomputation.

This perceive–plan–act–verify loop with tool use, memory and human checkpoints is what distinguishes an agentic system from a plain LLM wrapper. MedSphere therefore implements its assistant as an orchestrated, multi-step agent workflow (LangGraph) sitting on top of a GNN-scored knowledge graph.

1.3 Project Objectives

The project aims to build an intelligent, data-driven KOL Insights platform that lets Medical Affairs teams identify relevant and influential KOLs, analyse their professional relationships and network influence, and understand their regional impact. The specific objectives are:

1. **Data integration** – unify heterogeneous biomedical sources (OpenAlex, ORCID, ClinicalTrials.gov, PubMed) into canonical researcher entities.

2. **Graph construction** – build a heterogeneous graph with four relation types (collaborates, researches, affiliated, investigates) and a live Neo4j knowledge graph.
3. **Machine learning** – train a multi-task GNN to predict specialization, region and disease expertise and to produce network-aware HCP embeddings.
4. **Referral and similarity prediction** – derive HCP-to-HCP referral recommendations from embedding similarity.
5. **Agentic GraphRAG assistant** – answer natural-language questions with ranked KOLs, scores and grounded explanations.
6. **Human oversight** – embed Human-in-the-Loop (HITL) and Human-on-the-Loop (HOTL) validation at the point of prediction.
7. **Interactive visualisation** – deliver a Streamlit application for search, profiling, network exploration and live query.

2. Problem Statement and Business Case

2.1 Problem Definition

Problem statement

Medical Affairs teams face difficulty in identifying and prioritising the right Key Opinion Leaders across therapeutic areas and regions, because KOL selection often relies on separate indicators such as publications, citations, referrals, clinical trials and collaborations. This makes it difficult to obtain a complete view of a KOL's expertise, influence and professional network, leading to inefficient KOL engagement, missed collaboration opportunities and ineffective allocation of Medical Affairs resources.

Three concrete pain points follow from this problem:

- **Fragmented evidence.** Publication counts, citation metrics, trial roles and affiliations live in different systems and are rarely combined into one profile.
- **Influence is relational.** Citation counts alone do not show who is well connected, who bridges communities or who is likely to be a referral hub; this needs network analysis.

2.2 Business and Real-World Context

KOL identification and mapping is an established commercial capability in the life-sciences industry. Existing platforms provide geographic and organisational KOL mapping, but referral-based influence scoring and evidence-backed explanations are not their main focus. The gap MedSphere addresses is a **referral-aware, graph-native and explainable** KOL intelligence layer.

Table 2.1 – Business value delivered to Medical Affairs teams

Value driver	How MedSphere delivers it
Find KOLs faster	Identify relevant HCPs by disease, specialty or region through search filters or natural-language queries.
Prioritise experts	Rank HCPs using GNN-derived influence, similarity and referral scores.
Evidence-based decisions	Every recommendation is supported by graph paths, publications, trials and expertise evidence retrieved by GraphRAG.
Save resources	Reduces manual KOL research and cross-source analysis.
Actionable insights	Returns ranked KOLs with scores and plain-language explanations that MSLs can act on.

3. User and Stakeholder Analysis

3.1 Target Users

MedSphere is designed for teams that plan and execute scientific engagement in pharmaceutical and healthcare organisations. The primary users are Medical Affairs leadership and Medical Science Liaisons; KOLs and HCPs are the subjects whose data is analysed rather than direct operators.

3.2 Stakeholder Identification

Table 3.1 Stakeholders and Roles

Stakeholder	Role	Interest / need
Medical Affairs Leadership	Sets engagement strategy and allocates resources	Regional and therapeutic-area view of influence; defensible, evidence-based prioritisation; efficient use of budget
Medical Science Liaisons (MSLs)	Engage KOLs day to day	Quickly find the right experts, understand their networks and prepare outreach with clear explanations
KOLs / Healthcare Professionals	Subjects of analysis; collaborators	Accurate representation of their expertise; use of public, professional data only
Data / Platform team	Maintains pipelines, graph and models	Reproducible pipeline, monitoring and a way to correct model errors
Academic / Industry mentors	Guide and evaluate the project	Technical soundness, novelty and business relevance

3.3 User Requirements

Table 3.1 – Functional and non-functional requirements

ID	Requirement	Addressed by
FR1	Search and filter HCPs by name, specialization, region and tier	HCP Search view
FR2	View a 360° profile: research output, disease expertise, trials, network	KOL Profile view
FR3	Explore collaboration networks and communities visually	Network Explorer, ego-network map
FR4	Ask questions in natural language and receive ranked, explained answers	AI Assistant (GraphRAG agent)
FR5	Inspect the evidence and query behind an answer	GraphRAG Debug Panel, Neo4j Live Query
FR6	Accept, reject or recompute a prediction	HITL / HOTL controls
FR7	Generate and export referral networks	Referral Network view, CSV export
NFR1	Explainability – scores must be traceable to evidence	Explanation panels, GraphRAG evidence
NFR2	Responsiveness – interactive filtering and rendering	Cached data, sparse CSR matrices
NFR3	Scalability to large HCP and relationship data	Graph database; 155,940-node training graph

4. Proposed Agentic AI Solution

4.1 Overall Solution

MedSphere is an AI-powered KOL intelligence platform designed specifically for Medical Affairs teams. It combines GNN-based influence scoring, GraphRAG, Neo4j, LangGraph and generative AI (Google Gemini) to identify, prioritise and explain the most relevant HCPs for a specific disease, therapeutic area or region.

The solution has three cooperating layers:

- **Knowledge layer** – a heterogeneous knowledge graph in Neo4j AuraDB holding HCPs, institutions, specializations, regions, diseases, publications and trials, plus machine-generated SIMILAR_HCP edges and GNN scores.
- **Intelligence layer** – a Heterogeneous GNN that turns HCP features and relationships into 128-dimensional, network-aware embeddings used for specialization, region and disease prediction and for similarity and referral ranking.
- **Agentic layer** – a LangGraph-orchestrated assistant that understands the question, generates and validates Cypher, retrieves graph evidence (GraphRAG) and lets Gemini produce a grounded, structured answer, with human review of the outcome.

4.2 Key Features

Table 4.2 – Featured AI Models

Feature	Description
Multi-dimensional KOL intelligence graph	Combines research, trials, expertise, collaborations and networks in one profile
GNN-based prediction	Generates HCP embeddings, influence and referral scores
GraphRAG evidence	Retrieves relationships, paths, trials and expertise as supporting evidence
Explainable AI	Gemini returns ranked KOLs with scores and clear explanations
Referral network	Top-K referral graph per specialization × region with export
HITL / HOTL	Accept / Reject / Recompute controls embedded at the point of prediction

4.3 Agent-Based Approach

The assistant is built as a graph of specialised steps (agents) coordinated by a LangGraph orchestrator. Each step has a narrow responsibility, reads from and writes to a shared workflow state, and hands over to the next step. The approach has four defining properties:

- **Tool use** – the agent acts on the world through tools: a Text2Cypher model, the Neo4j query executor, the GNN score store and a graph renderer.
- **Self-verification** – a validity gate checks the generated query; invalid queries loop back for regeneration.
- **Grounded generation** – Gemini answers only from the retrieved graph information plus the user query, reducing hallucination.
- **Human collaboration** – the agent pauses to ask for disambiguation and exposes accept/reject/recompute controls after each result.

Table 4.3 – How MedSphere answers the core challenges

Challenge for Medical Affairs	How MedSphere overcomes it
Measuring true KOL influence	The GNN analyses HCP features and network relationships to generate influence and referral scores
Explaining KOL recommendations	GraphRAG retrieves supporting evidence and relationship paths, while Gemini generates clear explanations

5. Dataset / Knowledge Sources

5.1 Data Sources

MedSphere is built entirely on public biomedical and provider data. Four sources are ingested through dedicated adapters to build the researcher graph; three further reference sources provide citation metrics, provider validation and specialty taxonomy.

Table 5.1 – Data sources, key fields and purpose

Source	Data collected	Important columns	Purpose
PubMed	Publications + authors (~80K+ articles)	PMID, title, year, author, affiliation, DOI, MeSH	Research activity
OpenAlex	Authors, publications, institutions (~100K+ works)	Work ID, author ID, ORCID, institution ID, topics	Researcher and institution mapping
ORCID	Researcher identity (~50K+ profiles)	ORCID, name, employment	Identity resolution
ClinicalTrials.gov	Trials + investigators (~5K+ trials)	NCT ID, condition, investigator, role	Clinical activity
iCite	Citation metrics	PMID, citation count, RCR	Research impact
NPPES	US HCP information	NPI, taxonomy, city, state, ZIP	HCP validation and geography
NUCC	Medical taxonomy	Code, grouping, classification, specialization	Specialty mapping

5.2 Data Processing Pipeline

Raw records are converted into a consistent, analysis-ready form by a multi-stage pipeline (ingestion/ and processing/ packages):

1. **Ingestion** – source adapters write researchers, publications, trials and institutions.
2. **Normalisation and entity extraction** – cleaning and standardising names, affiliations and identifiers.
3. **Identity resolution** – merging records of the same person across sources into canonical person IDs (for example `canonical_openalex_...`, `canonical_orcid_...`).
4. **Taxonomy mapping** – mapping to 10 specializations, 8 disease categories and 3 geographic regions using YAML configuration.
5. **Relationship extraction and CSV export** – producing the tables used for graph construction.

5.3 Exploratory Data Analysis and Features

EDA is organised in three phases whose outputs become node features for the GNN.

Table 5.2 – EDA Steps

Phase	Purpose	Main activities
1. Influence analysis	How influential is the HCP based on research impact?	H-index, citations, RCR, publication count and recent activity → influence features
2. Expertise analysis	Which diseases / therapeutic areas is the HCP truly expert in?	Disease-related publications and trial activity; SapBERT for medical concepts and SPECTER2 for paper semantics → disease-expertise features
3. Network analysis	How well connected and important is the HCP?	Build collaboration network, centrality metrics, community detection → graph features

The resulting **48-dimensional feature vector** per researcher (standardised with StandardScaler) is summarised below.

Table 5.3 – Data Dimensions and Structure Analogy

Feature category	Dims	Examples
Publication metrics	8	Citation count, h-index, publication count, recency
Network centrality	6	Degree, betweenness, closeness, PageRank
Collaboration	4	Co-author count, collaboration diversity
Institutional	4	Institution rank, prestige score
Disease / topic	12	Topic distribution, disease relevance
Temporal	4	Career stage, activity trend
Geographic	4	Region encoding, international collaboration
Trial participation	6	Trial count, phase distribution, role

5.4 Dataset Statistics

Table 5.4 – Data Graph

Entity / relation	Count	Notes
Researcher (HCP) nodes	155,940	Primary nodes from four sources
Institutions / Publications / Trials	~5,000 / ~100,000 / ~5,000	Approximate
collaborates edges	274,912	Co-authorship
researches edges	147,658	Disease-relevance self-loops
affiliated edges	2,117,374	Institution co-affiliation
investigates edges	10,948	Trial co-investigator
Total directed edges	2,551,892	Four relation types

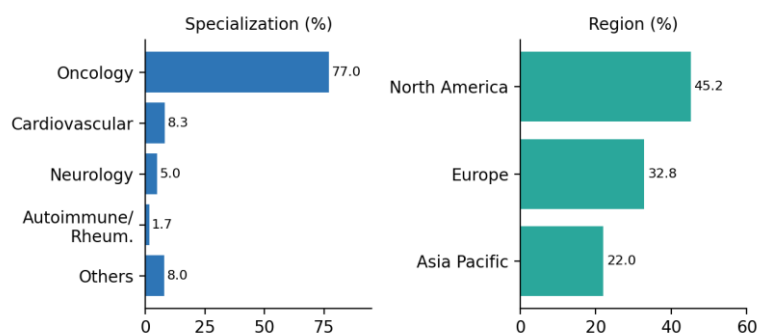


Figure 5.1 – Class distribution of the dataset (specialization is dominated by Oncology at 77%)

6. Tools and Technologies Used

Table 6.1 – Technology stack by layer

Layer	Technologies	Role in MedSphere
Data collection	Python 3.8+, Requests, JSON, CSV	Adapters for OpenAlex, ORCID, ClinicalTrials.gov, PubMed
Data processing	Pandas, NumPy, YAML config	Normalisation, identity resolution, taxonomy mapping
Language models for EDA	SapBERT, SPECTER2 (Hugging Face)	Disease-concept and paper-semantic features
Machine learning	PyTorch (sparse CSR tensors), scikit-learn	Heterogeneous GNN, training and evaluation
Graph database	Neo4j AuraDB, Cypher	Central knowledge graph; stores nodes, relationships, GNN scores and embeddings
Agent orchestration	LangGraph, LangChain	Stateful workflow controlling the assistant
Query translation	Hugging Face Text2Cypher model	Natural language → Cypher
Generative AI	Google Gemini	Intent understanding, grounded synthesis and explanations
Graph analytics & visualisation	NetworkX, PyVis, Plotly, Matplotlib	Network metrics, interactive topology maps and charts
Web application	Streamlit	Seven-view multi-page UI with a custom light theme
Engineering	Git, YAML/JSON configuration, unit tests	Reproducibility and validation

6.1 Why These Choices

- **Neo4j** stores relationships natively, so multi-hop questions (“who collaborates with whom, through which institution or trial”) are direct traversals rather than expensive joins.
- **A custom PyTorch heterogeneous GNN with CSR matrices** trained on CPU in 23.2 minutes; CSR conversion was about 15× faster than COO for the message-passing step (0.24 s vs 3.51 s per five passes on the affiliation matrix).
- **LangGraph** provides explicit state, conditional edges (the valid / invalid gate) and human-in-the-loop pauses, which are hard to obtain from a linear prompt chain.
- **Gemini** is used only where language understanding or generation is needed; ranking and scoring stay in the GNN, keeping numbers deterministic and auditable.

7. Agent Architecture

7.1 System Architecture Overview

Figure 7.1 shows the end-to-end architecture. Data sources flow through EDA into processed CSV data; a graph is constructed and the GNN trained; results are stored in Neo4j; the LangGraph orchestrator, Gemini and Text2Cypher connect the user interface to the knowledge graph.

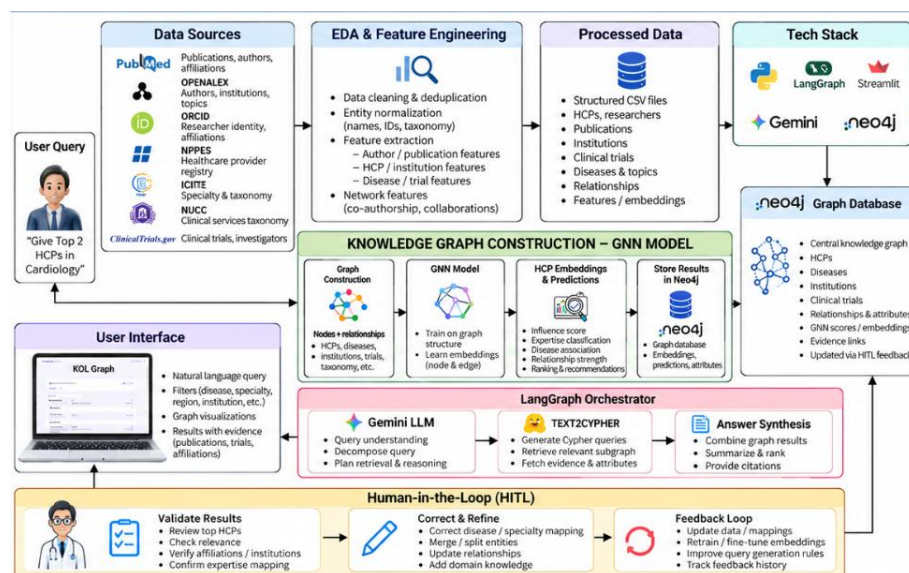


Figure 7.1 – MedSphere system architecture (from the project presentation)

7.2 Agent Components

The conversational assistant is decomposed into the components below. Each is implemented as a node of the LangGraph workflow in `service/graph_rag/workflow.py`.

Table 7.1 – Tech Stack and Use case

Agent / component	Responsibility	Model / tool
LangGraph Orchestrator	Receives the query, holds workflow state, controls routing, retries and pauses	LangGraph
Query Understanding agent	Intent classification; extraction of entities (HCP name, specialization, region, disease), filters and ranking needs	Gemini via LangChain
Entity-resolution step	Detects ambiguous names, pauses and asks the user to pick the correct HCP	Graph lookup + human input
Text2Cypher agent	Converts the understood query and schema context into an optimised Cypher query	Hugging Face Text2Cypher model
Query validator	Checks the Cypher; “NO” loops back to understanding / generation, “YES” proceeds	Rule and schema checks
GraphRAG retriever	Executes Cypher on Neo4j and packages nodes, edges, paths, trials, expertise and GNN scores	Neo4j driver
Synthesis / explanation agent	Writes the final structured answer and explanation from query + retrieved graph information	Gemini
Visualisation step	Renders returned nodes and edges as an interactive Plotly network	Plotly, NetworkX
Human oversight controls	Accept / Reject / Recompute after each prediction	Streamlit UI + feedback store

7.3 Tools

- **Text2Cypher generator** – schema-aware translation of language into graph queries.
- **Neo4j Cypher executor** – runs read queries (and, under human control, the recompute transaction).
- **GNN score store** – GNN rank, tier, embedding similarity (SIMILAR_HCP.embedding_similarity), specialization, region and disease predictions stored on the graph and in CSV artifacts such as region_predictions.csv, specialization_prediction and top_referral.csv.
- **Graph renderer** – PyVis / Plotly / NetworkX for topology views.
- **Validation and export utilities** – eight per-edge referral checks and three-file CSV export.

7.4 Memory

Table 7.2 – Memory and Graphical Understanding

Memory type	Implementation	Purpose
Working / short-term	LangGraph workflow state (query, extracted entities, generated Cypher, results); _HCP_RESOLUTION_CACHE for disambiguation choices	Carries context between steps within a turn and avoids re-asking the user to resolve the same HCP
Long-term knowledge	Neo4j knowledge graph with HCPs, diseases, institutions, trials, relationships and GNN scores / embeddings	Persistent, queryable domain memory shared by every session
Semantic memory	128-dimensional GNN embeddings (155,940 × 128) and SapBERT / SPECTER2 features	Similarity search and expertise matching
Feedback memory	Feedback store written by Accept / Reject actions	Records human judgements for continuous validation

7.5 Reasoning Flow

For each request the agent follows a plan – act – observe – verify – respond cycle:

1. **Understand** the request (intent, entities, filters, ranking need).
2. **Resolve** ambiguous entities with the user.
3. **Plan and act** – generate Cypher with the Text2Cypher tool and schema context.
4. **Verify** – if the query is invalid, return to step 1/3; otherwise execute it.
5. **Observe** – receive nodes, edges and scores from Neo4j.
6. **Respond** – combine query + retrieved graph information in Gemini to produce the final structured answer and a network visual.
7. **Review** – the user accepts, rejects or requests recomputation (HITL / HOTL).

7.6 Agent Interactions

Components interact through the shared workflow state rather than by calling each other directly. The orchestrator invokes the understanding agent, then (conditionally) the resolution step, the Text2Cypher agent and the validator; it then calls the retriever and finally the synthesis agent. The human is a participant at two points – disambiguation before retrieval and validation after the answer – and the user interface communicates with the orchestrator in both directions (Figure 7.1, bottom of diagram).

8. Agent Workflow and Orchestration

8.1 Workflow Design

Figure 8.1 shows the user-query workflow. A question enters “Retrieval & Query Routing”, where the LangGraph orchestrator, query-understanding step and the Text2Cypher model prepare a query. A **VALID?** gate decides whether the query proceeds to the graph database; if not, control returns to query understanding. Retrieved graph information is combined with the original query in GraphRAG and passed to Gemini, which returns the final structured answer.

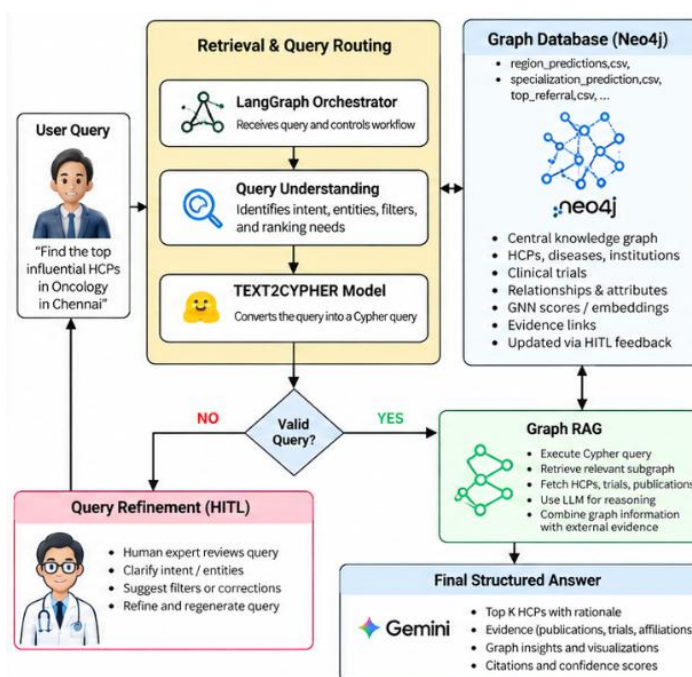


Figure 8.1 – User-query workflow: routing, validation loop, GraphRAG and answer generation (from the project presentation)

8.2 Task Decomposition

The example query “Find the top influential oncology HCPs in Chennai” is decomposed as follows:

Table 8.1 – Agentic Operation and Expected Output

Step	Sub-task	Output
1	Intent and entity extraction	Intent = ranked search; specialization = Oncology; location filter = Chennai; ranking = influence
2	Ambiguity check	No HCP name → no disambiguation needed
3	Cypher generation	Query filtering HCPs by specialization and region, ordered by GNN rank / score, with a LIMIT
4	Validation	Syntax and schema check; retry on failure
5	Execution and retrieval	Ranked HCPs with institutions, scores, relationships, trials and expertise
6	Synthesis	Table of ranked KOLs + “What this means” explanation
7	Visual and review	Interactive network plus Accept / Reject / Recompute

Regional filters in the prototype operate on the three region classes in the graph (North America, Europe, Asia Pacific); city-level filtering such as “Chennai” depends on the geographic granularity present in the loaded data.

8.3 Agent Coordination

- **Sequential hand-off** with conditional branches: the validator's outcome selects retry versus retrieval; the resolver's outcome selects "ask user" versus continue.
- **Shared state** holds the query, extracted entities, schema context, Cypher text, retrieved rows and answer, so every step sees the same facts.
- **Human coordination** – the workflow pauses for HCP selection and then resumes with the choice stored in the resolution cache.

8.4 Orchestration Implementation

The workflow is implemented in `service/graph_rag/workflow.py`. Conceptually it is a LangGraph state machine with the nodes and edges below (simplified pseudo-code):

```
graph = StateGraph(WorkflowState)
graph.add_node("understand", classify_intent_and_extract_entities)
graph.add_node("resolve", resolve_ambiguous_hcp)
graph.add_node("text2cypher", generate_cypher)
graph.add_node("retrieve", run_cypher_on_neo4j)
graph.add_node("synthesize", write_answer_with_gemini)

graph.set_entry_point("understand")
graph.add_edge("understand", "resolve")
graph.add_edge("resolve", "text2cypher")
graph.add_conditional_edges("text2cypher", is_valid,
                           {"YES": "retrieve", "NO": "understand"})
graph.add_edge("retrieve", "synthesize")
app = graph.compile()
```

The Streamlit AI Assistant page invokes this compiled workflow, renders the answer, shows a **GraphRAG Debug Panel** with the retrieved rows, draws the Plotly graph and then displays the HITL / HOTL controls.

8.5 Human-in-the-Loop and Human-on-the-Loop

Oversight is embedded directly beneath predictions in the AI Assistant and Neo4j Live Query views rather than in a separate dashboard:

Table 8.2 – Human in the Loop

Control	Effect
Accept	Confirms the model output and logs the judgement to the feedback store
Reject	Flags the prediction as a hallucination or data error
Recompute (HOTL)	Runs a live Cypher transaction that updates <code>SIMILAR_HCP.embedding_similarity</code> on the graph so that human intervention changes the stored weights

9. Agentic RAG Implementation

9.1 Retrieval Architecture

MedSphere uses **GraphRAG**: instead of retrieving text chunks from a vector index, the retriever queries a knowledge graph and returns structured evidence – ranked HCPs, relationship paths, trials, expertise and GNN scores. The retrieval architecture has four parts:

Table 9.1 – Chat Agent Retrieval

Component	Description
Knowledge base	Neo4j graph: HCP, Institution, Specialization, Region, Disease, Publication and Trial nodes; WORKS_AT, SPECIALIZES_IN, LOCATED_IN, EXPERTISE_IN, AUTHORED, INVOLVED_IN_TRIAL and SIMILAR_HCP relationships
Query planner	Gemini extracts intent and entities; the Text2Cypher model produces a Cypher query using the graph schema
Retriever	Neo4j executes the validated query and returns nodes, edges and properties as JSON
Context builder	Combines the user query with the retrieved graph information for generation

Graph schema

```
(HCP)-[:WORKS_AT]->(Institution)  (HCP)-[:AUTHORED]->(Publication)
(HCP)-[:SPECIALIZES_IN]->(Specialization) (HCP)-[:INVOLVED_IN_TRIAL]->(Trial)
(HCP)-[:LOCATED_IN]->(Region)      (HCP)-[:EXPERTISE_IN]->(Disease)
(HCP)-[:SIMILAR_HCP {embedding_similarity: float}]->(HCP) // GNN-generated
```

9.2 Knowledge Retrieval

Retrieval combines **symbolic** and **learned** signals. Cypher filters and traversals supply exact constraints (specialization, region, tier, institution, collaborators); the GNN-derived properties supply ranking (gnn_rank, KOL_tier, embedding_similarity). For example, the deck's Cypher illustration for top KOLs is:

```
MATCH (h:HCP)
```

```
WHERE h.specialization = 'Oncology'
```

```
RETURN h ORDER BY h.gat_score DESC LIMIT 10
```

and the referral network is retrieved with:

```
MATCH (s:HCP)-[:PREDICTED_REFERRAL]->(t:HCP)
```

```
WHERE s.specialization = $spec AND s.region = $region
```

```
RETURN s, r, t ORDER BY r.similarity DESC LIMIT $top_k
```

Retrieval quality is protected by three mechanisms: (i) the **VALID?** gate on generated Cypher, (ii) the entity-resolution step that avoids retrieving the wrong HCP, and (iii) referral-data validation (only edges with edge_type = PREDICTED_REFERRAL, edge_status = predicted, node_exists = True, finite similarity, positive rank and no self-loops are returned to the UI).

9.3 Generation Process

The generation step receives the **user query plus the retrieved graph information** and must answer from that evidence. The output format is a ranked table followed by a plain-language “What this means” section, with a collapsible GraphRAG Debug Panel exposing the retrieved rows. Explanation

prompts (for example “Why is Dr. T. Smith ranked highly?”) return the HCP profile, GNN rank, embedding similarity and similar HCP connections. Importantly, the interface states that the similarity value is a **cosine similarity between learned GNN embeddings** and does not represent a probability of collaboration, referral or future interaction – a deliberate guard against over-interpretation.

9.4 Retrieval-Quality Evaluation

Two recommendation pipelines were evaluated with ranking metrics at cut-off 10: **Pipeline A** ranks by the GNN alone, while **Pipeline B** uses GraphRAG with generative AI. Table 9.1 lists the approximate values read from the evaluation charts (Figure 11.5).

Table 9.2 – Recommendation quality of the two pipelines (approximate, read from charts)

Metric	Pipeline A (GNN)	Pipeline B (GraphRAG + GenAI)
Precision@10	≈ 0.062	≈ 0.054
Recall@10	≈ 0.042	≈ 0.036
NDCG@10	≈ 0.200	≈ 0.139
MRR	≈ 0.170	≈ 0.118

Pipeline A scores higher on every ranking metric, which is expected: it ranks directly on learned embeddings, whereas Pipeline B adds a language-driven retrieval and generation stage that trades some ranking accuracy for **explanations, natural-language access and evidence traceability**. Absolute values are low, which reflects the difficulty of recovering a specific relevant set from a very large candidate pool of 155,940 researchers; these figures should be read as a comparative baseline rather than a production accuracy claim.

10. System Implementation

10.1 Application Overview

The prototype is a multi-page Streamlit application (ui/app.py) backed by a Python service layer and Neo4j AuraDB. It offers seven views:

10.2 Functional Prototype Screenshots

The following screenshots were captured from the running prototype.

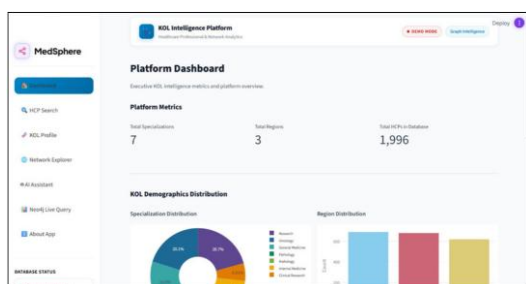


Figure 10.1 – Platform Dashboard with live metrics and demographic distributions

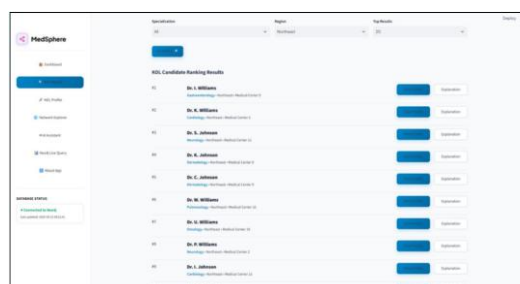


Figure 10.2 – HCP Search: ranked KOL candidates with explanation buttons

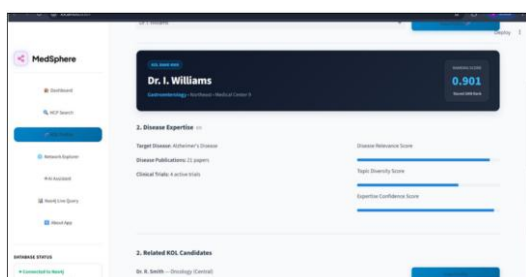


Figure 10.3 – KOL Profile: ranking score and disease-expertise scores



Figure 10.4 – Network Explorer: 2D collaboration graph and node details

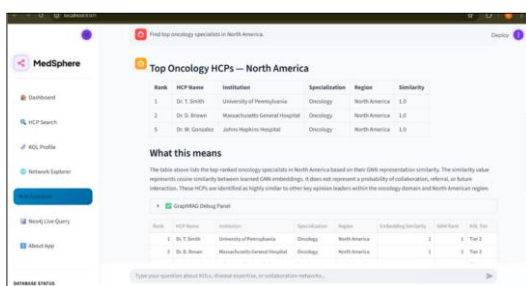


Figure 10.5 – AI Assistant: ranked oncology HCPs with explanation and GraphRAG debug panel

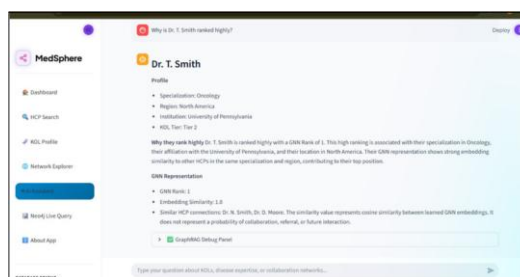


Figure 10.6 – AI Assistant: “Why is Dr. T. Smith ranked highly?”

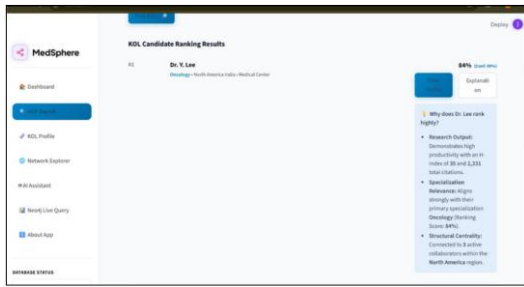


Figure 10.7 – Explanation panel: research output, specialization relevance, structural centrality



Figure 10.8 – Ego-network topology map for a selected researcher

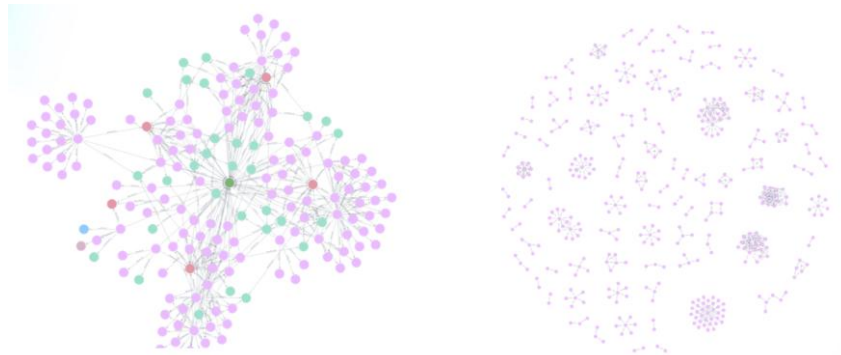


Figure 10.9 – Neo4j graph database output: connected HCP clusters (left) and community structure (right)

11. Experimental Evaluation and Results

11.1 Evaluation Metrics

- **Classification (GNN):** accuracy, macro-F1, weighted-F1, macro precision / recall and confusion matrices for specialization, region and disease.
- **Ranking / recommendation:** Precision@10, Recall@10, NDCG@10 and MRR for the two pipelines.
- **Data-integrity checks:** eight per-edge validation checks on referral output.
- **Efficiency:** training time, time per epoch, memory and interactive response time.

11.2 Testing Methodology

Table 11.1 – Testing Method

Item	Setting
Split	70 / 15 / 15 – 109,158 train, 23,391 validation, 23,391 test nodes; stratified by specialization × region; test set untouched during training
Model	2-layer relation-aware message passing, 128-d L2-normalised embedding, three task heads (10 / 3 / 8 classes), 229,653 parameters
Optimisation	AdamW, lr 1e-3, weight decay 1e-4, gradient clipping 1.0, ReduceLROnPlateau, early stopping (15 epochs), loss weights 1.0 / 1.0 / 0.5

Class imbalance	Inverse-frequency class weights capped at 10× (e.g., Oncology 0.13; rare classes 10.0)
Hardware	Intel i7, 16 GB RAM, CPU only
System testing	Referral-service tests, <code>validate_referral_data.py</code> , and functional testing of the seven UI views against the Neo4j backend

11.3 Performance Results

Table 11.2 – Test-set results

Task	Accuracy	Macro F1	Weighted F1
Specialization (10 classes)	72.82%	0.234	0.762
Region (3 classes)	75.76%	0.742	0.760
Disease (8 classes)	75.95%	0.290	0.800
Average accuracy	74.85%		

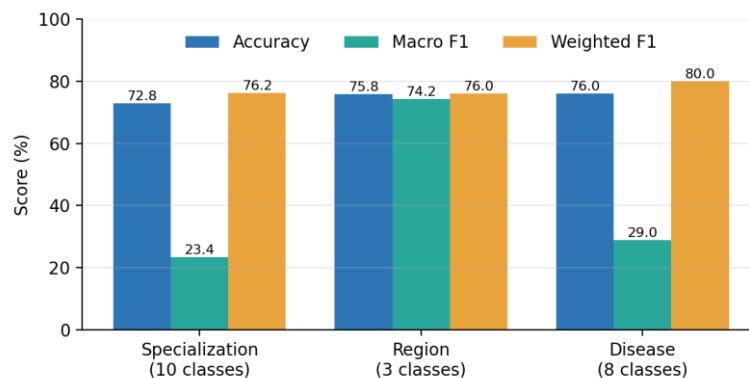


Figure 11.1 – GNN test performance across the three tasks

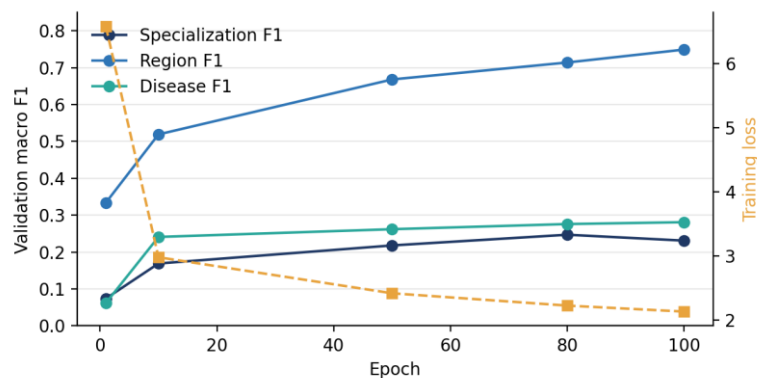


Figure 11.2 – Training progress at key epochs (loss and validation F1)

Region classification

Region is the most balanced task (macro F1 0.742 $\approx$ weighted F1 0.760). The confusion matrix has a strong diagonal; the largest confusion is Asia Pacific predicted as North America (609 nodes).

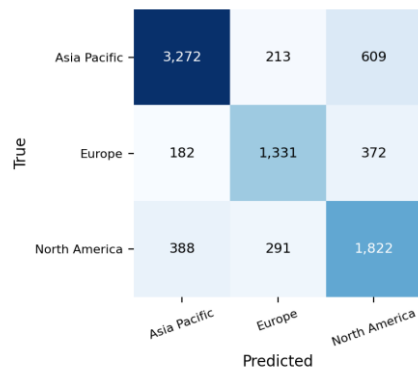


Figure 11.3 – Region confusion matrix (test set)



Figure 11.4 – Specialization confusion matrix (from the presentation)

Interpreting the class-imbalance effect

Oncology accounts for 77% of specialization labels, so a trivial classifier that always predicts Oncology would reach roughly 77% accuracy – higher than the model’s 72.8%. The GNN was deliberately trained with class-weighted loss so that it learns to recognise minority specializations, which lowers raw accuracy but raises minority-class recall. This is why weighted and macro F1 are reported alongside accuracy. The low macro F1 (0.234 for specialization, 0.290 for disease) reveals that the rarest classes – for example Rare Diseases (10 samples), Respiratory (18) and Endocrine / Metabolic (22) – are still predicted poorly, and is the main modelling limitation.

Referral network quality

Predicted referral candidates have high semantic similarity (median 0.945, 95th percentile 0.975, range 0.930–0.998). In the test run, 13.3% of predicted referrals coincide with an observed graph edge, indicating that the model recovers existing relationships while also proposing new candidates. All eight validation checks passed for the default Oncology × North America configuration (1 source HCP, 2 targets, 2 edges, average similarity 0.966). Note that this run used only three source HCPs, so it demonstrates correctness of the pipeline rather than statistical performance.

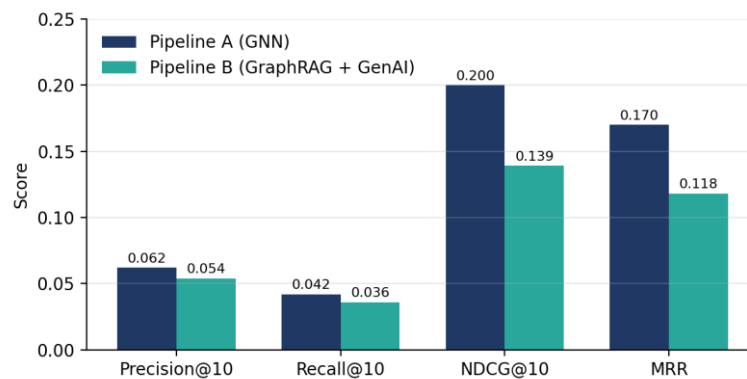


Figure 11.5 – Recommendation quality: Pipeline A (GNN) vs Pipeline B (GraphRAG + GenAI); values approximate

11.4 Efficiency Results

Table 11.3 – Result Efficiency

Measure	Result
Total training time	23.2 minutes for 100 epochs (CPU-only)
Time per epoch	≈ 20 s cold start, ≈ 13 s warm
CSR vs COO speed-up	15× (0.24 s vs 3.51 s per 5 passes)
Peak memory	≈ 2.5 GB RAM
Embedding extraction	1.2 s for all 155,940 researchers
Referral generation (top-5, 3 sources)	8.5 s
UI network filter + render (50 nodes, 100 edges)	0.5 s

12. Comparison, Innovation and Scalability

12.1 Comparison with Baseline

Two comparisons are relevant: against existing commercial KOL platforms (qualitative), and against the internal GNN-only baseline (quantitative, Section 9.4).

Table 12.1 – Existing KOL platforms and their limitations

Platform	Capability	Limitation relative to MedSphere
Konectar (Aissel)	KOL mapping at state, city and ZIP level	Broad KOL platform; referral-based influence is not the main focus
VML Health	Global, regional and local KOL mapping	Lacks a focused referral-network approach
MedView Insights	Regional influence maps and geographic analysis	Referral-graph analysis not specifically described publicly
ZS ZAIDYN	AI-driven KOL identification across geographies and therapeutic areas	Broad life-sciences platform; needs configuration for referral-based analysis
Acuity (81qd)	KOL mapping across organisations and regions	Strong mapping; limited focus on referral-based influence scoring

This comparison is based on publicly available descriptions of these products; it is not a head-to-head benchmark.

Table 12.2 – Benchmark achieved

Aspect	Manual / siloed approach	GNN only (Pipeline A)	MedSphere (agentic GraphRAG)
Signals combined	One indicator at a time	Features + network	Features + network + evidence paths
Access	Analyst effort across tools	Scores / tables	Natural-language questions
Explanations	Manual	None built-in	Evidence-based, plain-language
Self-checking	Not applicable	Not applicable	Query validation loop, human review

Ranking accuracy (NDCG@10)	Not measured	≈ 0.20	≈ 0.14
---------------------------------------	--------------	----------------	----------------

12.2 Advantages of the Proposed System

- **Complete view of a KOL** – expertise, influence and network in one graph-native profile.
- **Explainability** – ranked KOLs come with scores, relationship paths and natural-language reasoning.
- **Natural-language access** – non-technical Medical Affairs users can query the graph without writing Cypher, while analysts can still use Neo4j Live Query.
- **Trust and safety** – no fabricated persons or edges, an explicit similarity-is-not-probability caveat, and human accept / reject controls.
- **Low cost to run** – trains on CPU in under 25 minutes with about 2.5 GB memory.

12.3 Innovation

- **Multi-dimensional KOL intelligence** combining research, trials, expertise, collaborations and networks.
- **Referral-centric influence scoring** with a heterogeneous GNN using config-driven relation weights (collaborates 1.0, researches 1.0, affiliated 0.743, investigates 0.8).
- **GraphRAG plus GNN scores** – retrieval returns learned scores together with explicit relationship paths as evidence.
- **Agentic workflow with validation loop and embedded HITL / HOTL** rather than a stand-alone chatbot.

12.4 Scalability

- The graph-based architecture can absorb new specialties, regions, hospitals and therapeutic areas and new data sources (conferences, digital activity) without re-architecting.
- The pipeline already processes 155,940 researchers and 2.55 million relationships on commodity CPU hardware; sparse CSR operations and cached UI data keep interaction responsive.
- Neo4j supports continuous updates as professional relationships and influence change; incremental learning and graph sampling are planned for larger scale.

12.5 Real-World Applicability

Pharmaceutical companies can identify KOLs for advisory boards, trials and speaker programmes; medical societies can discover emerging leaders; healthcare organisations can map professional networks; and researchers can locate domain experts and collaborators. Before production use the team would need to address the limitations listed in Section 13.2.

13. Conclusion and Future Work

13.1 Conclusion

MedSphere demonstrates an end-to-end agentic AI platform for KOL intelligence. Public biomedical data from four sources was unified into a graph of 155,940 researchers and 2.55 million relationships; a heterogeneous GNN trained on CPU in 23.2 minutes reached 72.8–76.0% accuracy and 0.76–0.80 weighted F1 across specialization, region and disease tasks; and a LangGraph-orchestrated GraphRAG assistant on Neo4j turns those scores into ranked, explained and visualised recommendations, with human validation built into the workflow. The Streamlit prototype meets all 23 Phase-3 acceptance criteria and gives Medical Affairs teams a practical way to find, prioritise and justify KOL engagement decisions.

13.2 Limitations

- **Rare-class performance.** Macro F1 is low for specialization (0.234) and disease (0.290); rare specializations remain poorly predicted despite class weighting.
- **Ranking accuracy.** Precision@10 (≈ 0.06) and NDCG@10 (≈ 0.20) are modest, and the GraphRAG pipeline scores below GNN-only ranking.
- **Prototype data scale.** The UI dashboard was demonstrated with a demo Neo4j instance (about 2,000 HCPs, 7 specializations, 3 regions), far smaller than the 155,940-researcher training graph; some screenshots use synthetic names.
- **Recompute mechanism.** The HOTL “Recompute” action currently updates similarity weights with a Cypher expression that produces randomised values in [0.5, 0.9]; it demonstrates human-controlled mutation but should be replaced by genuine re-inference or feedback-driven retraining.
- **Interpretation of similarity.** Embedding similarity is not a probability of referral or collaboration; it must be complemented with real referral data when available.
- **Geographic granularity.** The model predicts three broad regions; city-level filtering depends on additional NPPEs-derived geography.

13.3 Future Work

Table 13.1 – Future Work

Horizon	Planned work
Short term (1–3 months)	Real researcher name resolution; institution metadata enrichment; MeSH / SNOMED CT disease ontologies; GraphSAGE baseline comparison; attention visualisation and SHAP explanations; transfer learning for rare specializations; ranked-list and network-comparison views
Medium term (3–6 months)	FastAPI backend with authentication and rate limiting; Docker and CI/CD; on-demand GNN inference; streaming ingestion; community detection and temporal network evolution; feedback loop that retrains on Accept / Reject data
Long term (6–12 months)	Emerging-KOL detection and real-time monitoring; predictive analytics for future influence; multi-GPU training and incremental learning; advisory-board and speaker-bureau recommendations; CRM and calendar integration; fairness, bias and privacy-preserving analysis